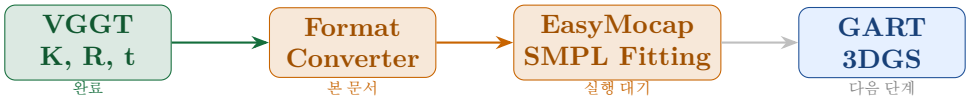
Integration Guide

VGGT + EasyMocap 통합 파이프라인

VGGT 카메라 캘리브레이션 결과를 EasyMocap SMPL 피팅에 활용하는 방법론, 이론, 구현



2026 년 4 월 12 일

VGGT (CVPR 2025)의 카메라 파라미터를 EasyMocap의 intri.yml / extri.yml 포맷으로 변환하여, 기존에 좌표계 불일치로 실패했던 SMPL 피팅을 재시도합니다.

Contents

1 문제 정의와 전략 **2**
1.1 이전 실패 원인 요약 2
1.2 새 전략: VGGT → EasyMocap 2

2 이론: 좌표계와 변환 **4**
2.1 VGGT의 좌표 규약 4
2.2 EasyMocap의 좌표 규약 4
2.3 투영 모델 4
2.4 EasyMocap의 삼각측량 5
2.5 EasyMocap의 SMPL 피팅 5

3 개발 방향 **6**
3.1 Phase 1: VGGT → EasyMocap 포맷 변환 6
3.2 Phase 2: EasyMocap 실행 7
3.3 Phase 3: 검증 7
3.4 Phase 4: 스케일 보정 (필요시) 7

4 구현 코드 **8**
4.1 VGGT → EasyMocap 변환 스크립트 8
4.2 실행 스크립트 10
4.3 검증 스크립트: 재투영 오차 11

5 개발 방향 로드맵 **13**
5.1 잠재적 문제와 대비책 13

6 기대 효과 **15**

Chapter 1

문제 정의와 전략

1.1 이전 실패 원인 요약

Warning

이전 EasyMocap SMPL 피팅이 실패한 핵심 원인:

1. DA3의 K가 280×280에서 추정되어, 원본 해상도로의 역변환 과정에서 $s_x \neq s_y$ 발생
2. 역변환된 K와 OpenPose 2D 키포인트의 좌표계 불일치
3. 결과적으로 삼각측량된 3D 점과 SMPL 포즈가 다른 좌표 공간에 존재
4. EasyMocap 출력의 Y/Z 축 교환 현상 (Th의 좌표가 기대값과 불일치)

1.2 새 전략: VGGT → EasyMocap

Key Idea

VGGT의 K는 원본 해상도 좌표계에서 직접 매칭되므로, OpenPose 2D 키포인트와 좌표계가 자동으로 일치한다. 이 K와 R, t를 EasyMocap의 포맷으로 변환하면, 좌표계 불일치 문제가 구조적으로 해결된다.

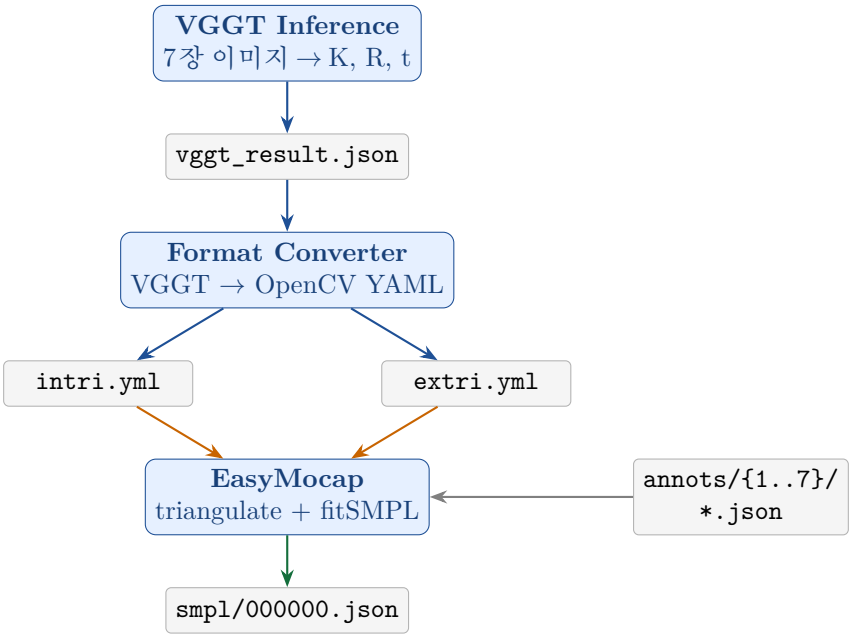


Figure 1.1: VGGT → EasyMocap 통합 파이프라인

Chapter 2

이론: 좌표계와 변환

2.1 VGGT의 좌표 규약

VGGT는 **OpenCV 규약**을 사용한다:

- x: 오른쪽, y: 아래쪽, z: 앞쪽 (카메라 시선 방향)
- Extrinsic $[\mathbf{R} \mid \mathbf{t}]$는 **world-to-camera** 변환
- $\mathbf{p}_{\text{cam}} = \mathbf{R} \cdot \mathbf{p}_{\text{world}} + \mathbf{t}$

  카메라의 월드 좌표 위치는:

$$\mathbf{p}_{\text{cam_center}} = -\mathbf{R}^\top \mathbf{t} \tag{2.1}$$

2.2 EasyMocap의 좌표 규약

EasyMocap도 **OpenCV 규약**을 사용한다:

- `extri.yml`의 `Rot_X`: 3×3 회전 행렬 (world-to-camera)
- `extri.yml`의 `T_X`: 3×1 평행이동 벡터
- `extri.yml`의 `R_X`: Rodrigues 벡터 (3×1, 축-각도 표현)
- `intri.yml`의 `K_X`: 3×3 내부 행렬
- `intri.yml`의 `dist_X`: 1×5 왜곡 계수 $(k_1, k_2, p_1, p_2, k_3)$

Summary

VGGT와 EasyMocap은 같은 좌표 규약 (OpenCV)을 사용한다. 따라서 좌표 변환이 필요 없고, 포맷만 변환하면 된다.

2.3 투영 모델

두 시스템 모두 동일한 핀홀 투영 모델을 사용:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}}_{\mathbf{K}} \underbrace{\begin{pmatrix} \mathbf{R} & \mathbf{t} \end{pmatrix}}_{[\mathbf{R}|\mathbf{t}]_{3\times 4}} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} \tag{2.2}$$

여기서 (u, v)는 OpenPose가 검출한 2D 키포인트 좌표 (원본 해상도 기준). VGGT의 K가 원본 해상도 기준이므로, (u, v)와 K가 같은 좌표계에 있다.

2.4 EasyMocap의 삼각측량

EasyMocap의 `SimpleTriangulate`는 다음을 수행:

1. 각 카메라 i에서 투영 행렬 구성: $\mathbf{P}_i = \mathbf{K}_i[\mathbf{R}_i \mid \mathbf{t}_i]$
2. 2D 키포인트 (u_j^i, v_j^i)와 $\mathbf{P}_i$로 DLT 삼각측량
3. Iterative refinement: confidence 가중 + 재투영 오차 기반 outlier 제거
4. 결과: 3D 키포인트 $\mathbf{p}_j \in \mathbb{R}^3$ (월드 좌표)

Important

삼각측량의 정확도는 전적으로 **K와 2D 키포인트의 좌표계 일치**에 의존한다. 이전에 DA3의 K가 틀려서 삼각측량이 부정확했고, 그 부정확한 3D 점으로 SMPL을 피팅하니 실패한 것이다. VGGT의 K를 사용하면 이 문제가 해결된다.

2.5 EasyMocap의 SMPL 피팅

삼각측량된 3D 키포인트로부터 SMPL 파라미터를 추정:

$$\min_{\boldsymbol{\theta},\boldsymbol{\beta},\mathbf{R}_h,\mathbf{t}_h} \underbrace{\sum_j w_j \|J_j(\boldsymbol{\theta},\boldsymbol{\beta}) - \mathbf{p}_j\|^2}_{\text{3D joint loss}} + \underbrace{\lambda_\theta \|\boldsymbol{\theta}\|^2}_{\text{pose prior}} + \underbrace{\lambda_\beta \|\boldsymbol{\beta}\|^2}_{\text{shape prior}} \tag{2.3}$$

여기서:

- $J_j(\boldsymbol{\theta}, \boldsymbol{\beta})$: SMPL의 forward kinematics로 계산된 관절 위치
- $\mathbf{p}_j$: 삼각측량된 3D 키포인트 (25개, Body25 포맷)
- $\mathbf{R}_h, \mathbf{t}_h$: global rotation (Rodrigues), global translation
- $\boldsymbol{\theta} \in \mathbb{R}^{69}$: body pose (23 joints $\times$ 3 axis-angle)
- $\boldsymbol{\beta} \in \mathbb{R}^{10}$: body shape

EasyMocap의 추가 단계:

1. **Init**: 삼각측량된 pelvis 위치로 $\mathbf{t}_h$ 초기화
2. **Shape optimization**: $\boldsymbol{\beta}$ 먼저 최적화 (골격 비율)
3. **Pose optimization**: $\boldsymbol{\theta}$ 최적화
4. **Joint optimization**: 모든 파라미터 동시 최적화
5. (Optional) **2D reprojection refinement**: 각 카메라의 2D 키포인트에 대한 재투영 오차도 추가

Chapter 3

개발 방향

3.1 Phase 1: VGGT → EasyMocap 포맷 변환

변환 요구사항

1. VGGT JSON → `intri.yml`: 카메라별 K, dist (왜곡 = 0)
2. VGGT JSON → `extri.yml`: 카메라별 R (Rodrigues), Rot (회전행렬), T (평행이동)
3. 카메라 이름 매핑: cam1→"1", cam2→"2", ...

`intri.yml` 포맷:

```
%YAML:1.0
---
names: ["1", "2", "3", "4", "5", "6", "7"]
K_1: !!opencv-matrix
  rows: 3
  cols: 3
  dt: d
  data: [fx, 0, cx, 0, fy, cy, 0, 0, 1]
dist_1: !!opencv-matrix
  rows: 1
  cols: 5
  dt: d
  data: [0, 0, 0, 0, 0]
```

`extri.yml` 포맷:

```
R_1: !!opencv-matrix     # Rodrigues vector (3x1)
  rows: 3
  cols: 1
  dt: d
  data: [rx, ry, rz]
Rot_1: !!opencv-matrix   # Rotation matrix (3x3)
  rows: 3
  cols: 3
  dt: d
  data: [r00, r01, r02, r10, r11, r12, r20, r21, r22]
T_1: !!opencv-matrix     # Translation (3x1)
  rows: 3
  cols: 1
```

```
  dt: d
  data: [tx, ty, tz]
```

3.2 Phase 2: EasyMocap 실행

실행 명령

```cpp
cd ~/EasyMocap
python3 apps/demo/mv1p.py \
  ~/easymocap_data \
  --out ~/easymocap_data/output/vggt_smpl \
  --cfg config/exp/mv1p/detect_triangulate_fitSMPL.yml \
  --vis_det --vis_repro
```

3.3 Phase 3: 검증

1. **삼각측량 검증**: 재투영 오차 (reprojection error) 계산

$$e_{\text{reproj}} = \frac{1}{N \cdot C} \sum_{j=1}^{N} \sum_{i=1}^{C} \|\pi(\mathbf{K}_i, \mathbf{R}_i, \mathbf{t}_i, \mathbf{p}_j) - (u_j^i, v_j^i)\| \tag{3.1}$$

   목표: $e_{\text{reproj}} < 10$ px

2. **SMPL 피팅 검증**: MPJPE 계산

$$\text{MPJPE} = \frac{1}{J} \sum_{j=1}^{J} \|J_j^{\text{SMPL}} - \mathbf{p}_j^{\text{triangulated}}\| \tag{3.2}$$

   목표: MPJPE < 50 mm (이전 실패: 219.6 mm)

3. **시각적 검증**: SMPL 메시를 각 카메라 뷰에 재투영하여 오버레이

3.4 Phase 4: 스케일 보정 (필요시)

Warning

VGGT의 translation은 정규화된 좌표 ($\|t\| \sim 1$)이므로, SMPL의 크기가 현실과 다를 수 있다. SMPL의 pelvis 높이가 실제 피사체와 일치하도록 global scale factor를 곱해야 할 수 있다.

$$s = \frac{h_{\text{subject_cm}}/100}{\|J_{\text{head}} - J_{\text{ankle}}\|_{\text{SMPL}}} \tag{3.3}$$

Chapter 4

구현 코드

4.1 VGGT → EasyMocap 변환 스크립트

```python
#!/usr/bin/env python3
"""
Convert VGGT calibration JSON to EasyMocap intri.yml / extri.yml.

Usage:
  python vggt_to_easymocap.py \
    --input vggt_calibration_result.json \
    --output_dir ~/easymocap_data
"""

import json
import argparse
import numpy as np
import cv2
from pathlib import Path


def write_opencv_matrix(f, name, mat, rows, cols):
    """Write a matrix in OpenCV YAML format."""
    f.write(f"{name}: !!opencv-matrix\n")
    f.write(f"  rows: {rows}\n")
    f.write(f"  cols: {cols}\n")
    f.write(f"  dt: d\n")
    flat = mat.flatten().tolist()
    data_str = ", ".join(f"{v:.10f}" for v in flat)
    f.write(f"  data: [{data_str}]\n")


def rotation_matrix_to_rodrigues(R):
    """Convert 3x3 rotation matrix to Rodrigues vector."""
    rvec, _ = cv2.Rodrigues(np.array(R, dtype=np.float64))
    return rvec.flatten()


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", required=True,
                        help="VGGT calibration JSON")
```

```python
    parser.add_argument("--output_dir", required=True,
                        help="EasyMocap data directory")
    args = parser.parse_args()

    with open(args.input) as f:
        vggt = json.load(f)

    out = Path(args.output_dir)
    cam_names = sorted(vggt.keys(),
                       key=lambda x: int(x.replace("cam", "")))
    names = [c.replace("cam", "") for c in cam_names]

    # ---- Write intri.yml ----
    intri_path = out / "intri.yml"
    with open(intri_path, "w") as f:
        f.write("%YAML:1.0\n---\n")
        name_list = ", ".join(f'"{n}"' for n in names)
        f.write(f"names:\n")
        for n in names:
            f.write(f'  - "{n}"\n')

        for cam_key, name in zip(cam_names, names):
            cam = vggt[cam_key]
            K = np.array(cam["K"], dtype=np.float64)
            write_opencv_matrix(f, f"K_{name}", K, 3, 3)
            dist = np.zeros(5)
            write_opencv_matrix(f, f"dist_{name}", dist, 1, 5)

    print(f"Written: {intri_path}")

    # ---- Write extri.yml ----
    extri_path = out / "extri.yml"
    with open(extri_path, "w") as f:
        f.write("%YAML:1.0\n---\n")
        f.write(f"names:\n")
        for n in names:
            f.write(f'  - "{n}"\n')

        for cam_key, name in zip(cam_names, names):
            cam = vggt[cam_key]
            R_mat = np.array(cam["R"], dtype=np.float64)
            t_vec = np.array(cam["t"], dtype=np.float64)

            # Rodrigues vector from rotation matrix
            rvec = rotation_matrix_to_rodrigues(R_mat)

            write_opencv_matrix(f, f"R_{name}",
                                rvec.reshape(3, 1), 3, 1)
            write_opencv_matrix(f, f"Rot_{name}",
                                R_mat, 3, 3)
            write_opencv_matrix(f, f"T_{name}",
                                t_vec.reshape(3, 1), 3, 1)

    print(f"Written: {extri_path}")

    # ---- Verification ----
```

```python
    print("\n=== Verification ===")
    for cam_key, name in zip(cam_names, names):
        cam = vggt[cam_key]
        K = np.array(cam["K"])
        R = np.array(cam["R"])
        t = np.array(cam["t"])

        # Camera center in world coordinates
        center = -R.T @ t
        rvec = rotation_matrix_to_rodrigues(R)

        print(f"cam{name}: fx={K[0,0]:.1f} fy={K[1,1]:.1f} "
              f"| center=[{center[0]:.3f},{center[1]:.3f},"
              f"{center[2]:.3f}] "
              f"| rvec=[{rvec[0]:.4f},{rvec[1]:.4f},"
              f"{rvec[2]:.4f}]")

    # Rodrigues roundtrip check
    R_orig = np.array(vggt["cam2"]["R"], dtype=np.float64)
    rvec = rotation_matrix_to_rodrigues(R_orig)
    R_back, _ = cv2.Rodrigues(rvec)
    err = np.max(np.abs(R_orig - R_back))
    print(f"\nRodrigues roundtrip error: {err:.2e}")


if __name__ == "__main__":
    main()
```

Listing 4.1: vggt_to_easymocap.py — VGGT 결과를 EasyMocap intri.yml/extri.yml로 변환

4.2 실행 스크립트

```bash
#!/bin/bash
set -e

# === Step 1: Convert VGGT -> EasyMocap format ===
echo "[Step 1] Converting VGGT calibration to EasyMocap format..."
cd /home/elicer
python3 vggt_to_easymocap.py \
  --input /home/elicer/vggt_calibration_result.json \
  --output_dir /home/elicer/easymocap_data

# Backup old calibration
cp /home/elicer/easymocap_data/intri.yml \
   /home/elicer/easymocap_data/intri_da3_backup.yml 2>/dev/null || true
cp /home/elicer/easymocap_data/extri.yml \
   /home/elicer/easymocap_data/extri_da3_backup.yml 2>/dev/null || true

echo "[Step 1] Done. intri.yml and extri.yml updated."

# === Step 2: Run EasyMocap triangulation + SMPL fitting ===
echo "[Step 2] Running EasyMocap..."
cd /home/elicer/EasyMocap

```

```
python3 apps/demo/mv1p.py \
  /home/elicer/easymocap_data \
  --out /home/elicer/easymocap_data/output/vggt_smpl \
  --cfg config/exp/mv1p/detect_triangulate_fitSMPL.yml \
  --vis_det --vis_repro

echo "[Step 2] Done. Results in output/vggt_smpl/"

# === Step 3: Quick validation ===
echo "[Step 3] Checking SMPL output..."
ls -la /home/elicer/easymocap_data/output/vggt_smpl/smpl/
head -50 /home/elicer/easymocap_data/output/vggt_smpl/smpl/000000.json

echo "=== Pipeline complete ==="
```

Listing 4.2: run_vggt_easymocap.sh — 전체 파이프라인 실행

4.3 검증 스크립트: 재투영 오차

```
#!/usr/bin/env python3
"""
Validate triangulation quality by computing reprojection error.
"""

import json
import numpy as np
from pathlib import Path


def load_cameras_yml(intri_path, extri_path):
    """Load EasyMocap camera YAML files."""
    import cv2
    fs_intri = cv2.FileStorage(str(intri_path),
                               cv2.FILE_STORAGE_READ)
    fs_extri = cv2.FileStorage(str(extri_path),
                               cv2.FILE_STORAGE_READ)

    cameras = {}
    for i in range(1, 8):
        name = str(i)
        K = fs_intri.getNode(f"K_{name}").mat()
        dist = fs_intri.getNode(f"dist_{name}").mat()
        Rot = fs_extri.getNode(f"Rot_{name}").mat()
        T = fs_extri.getNode(f"T_{name}").mat()

        cameras[name] = {
            "K": K, "dist": dist,
            "R": Rot, "t": T.flatten()
        }

    fs_intri.release()
    fs_extri.release()
    return cameras

```

```python

def project_3d_to_2d(point3d, K, R, t):
    """Project a 3D point to 2D pixel coordinates."""
    p_cam = R @ point3d + t
    p_proj = K @ p_cam
    return p_proj[:2] / p_proj[2]


def compute_reprojection_error(cameras, kp3d, kp2d_per_cam):
    """Compute mean reprojection error across all cameras."""
    errors = []
    for cam_name, cam in cameras.items():
        if cam_name not in kp2d_per_cam:
            continue
        kp2d = kp2d_per_cam[cam_name]  # (N, 3): u, v, conf
        K = cam["K"]
        R = cam["R"]
        t = cam["t"]

        for j in range(len(kp3d)):
            if kp2d[j][2] < 0.3:  # low confidence
                continue
            proj = project_3d_to_2d(kp3d[j], K, R, t)
            err = np.linalg.norm(proj - kp2d[j][:2])
            errors.append(err)

    return np.mean(errors), np.median(errors), errors
```

Listing 4.3: validate_reprojection.py — 삼각측량 결과 검증

Chapter 5

개발 방향 로드맵

Table 5.1: 개발 로드맵

Phase	작업	성공 기준
1	VGGT → intri.yml / extri.yml 변환	포맷 정상, roundtrip 검증
2	EasyMocap 삼각측량 실행	reprojection error < 10 px
3	EasyMocap SMPL 피팅 실행	MPJPE < 50 mm
4	SMPL 메시 오버레이 시각화	시각적으로 인체에 정렬됨
5	스케일 보정 (필요시)	신장/사지 비율 현실적
6	GART 입력 준비 ($\boldsymbol{\theta}, \boldsymbol{\beta}$ per frame)	GART 학습 시작 가능
Fallback: Phase 2–3 실패 시		
F1	VGGT K로 직접 삼각측량 (EasyMocap 우회)	3D 키포인트 시각적 검증
F2	smplx 직접 피팅 (PyTorch, 서버 GPU)	MPJPE < 50 mm
F3	3DGS-Avatar JMBO (HMR 초기값 + joint opt.)	photometric loss 수렴

5.1 잠재적 문제와 대비책

1. **VGGT 스케일 문제**: VGGT의 translation이 정규화되어 있어 EasyMocap의 3D 재구성 스케일이 현실과 다를 수 있음.
   - 대비: 피사체 키 또는 어깨 너비로 스케일 보정
   - SMPL 피팅 자체는 상대적 관절 각도이므로 스케일 영향 적음
2. **EasyMocap의 SMPL 모델**: 기존에 `SMPL_NEUTRAL.pkl` (304KB 경량)을 사용하여 뼈 길이가 부정확했음.

- 대비: `SMPL_NEUTRAL.pkl` (full, ∼21MB)로 교체

3. **투구 동작의 극단적 포즈**: EasyMocap의 기본 pose prior가 일상 동작 기준이라 7,000 deg/s 어깨 회전에 적합하지 않을 수 있음.
   - 대비: `lambda_theta`를 0.0001로 낮추어 극단 포즈 허용
4. **왜곡 미보정**: VGGT는 왜곡 계수를 제공하지 않으므로, 이미지 가장자리의 키포인트에서 수 px 오차 발생 가능.
   - 대비: 투수가 이미지 중앙에 있으면 영향 최소 (< 5 px)
   - 필요시 OpenCV `calibrateCamera`로 사후 왜곡 추정 가능

Chapter 6

기대 효과

Summary

이전 실패 (DA3 K → EasyMocap)와 비교한 개선점:

1. **좌표계 일치**: VGGT K가 원본 해상도 ⇔ OpenPose 2D 키포인트 — 역변환 불필요, 불일치 위험 제거
2. f_x/f_y **정확도**: DA3 최대 2.2% 편차 → VGGT 최대 0.6% 편차
3. **다시점 일관성**: VGGT가 7장을 동시에 보고 R, t 추정 → 카메라 간 상대 위치가 더 일관적
4. **Y/Z 축 교환 해소**: DA3의 PnP 좌표계와 EasyMocap 좌표계 불일치 → VGGT는 OpenCV 규약으로 통일

이전 MPJPE 219.6 mm에서 **50 mm 이하**를 목표로 한다.